# ADVANCED PROGRAMMING TECHNIQUES

VNU - UNIVERSITY *of* ENGINEERING *&* TECHNOLOGY

# LECTURE 2: Programming File I/O in Linux

CONTENTS

# File

Computers store information on various persistent storage media


The OS *abstracts* from the physical devices to define a ***file*** = a logical storage unit

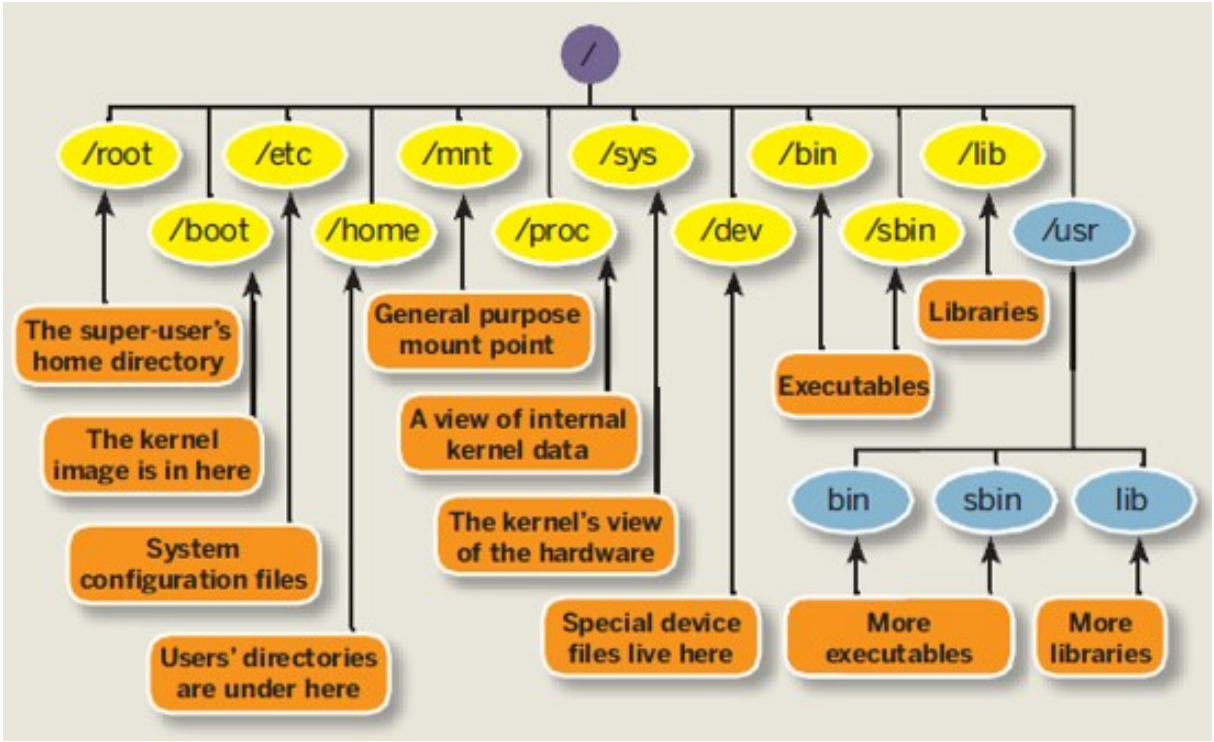
> Linux treats everything as a file, providing a consistent way (tools + commands) to interact with various system resources.
  - Regular file: a sequence of bytes stored on a medium, e.g. texts, images, etc.
  - Any object or HW device that reads/writes bytes
    - ✓ Terminals
    - ✓ Pipes
    - ✓ Sockets

# File Attributes

- Standard file attributes
  - Name: a file is a named collection of related information
  - Permissions: `read (r)/write (w)/execute (x)` rights of a file
  - Ownership: includes `owner (o)/group (g)/other (o)`
  - Time stamps: e.g., `atime/mtime/ctime`
  - Size
- Extended file attributes
  - E.g., immutable, append-only, no atime updates, etc.
  - Check extended attributes with the shell command: `lsattr filename`
- Special file attributes (for executables)
  - E.g., SetUID, SetGID, Sticky Bit.

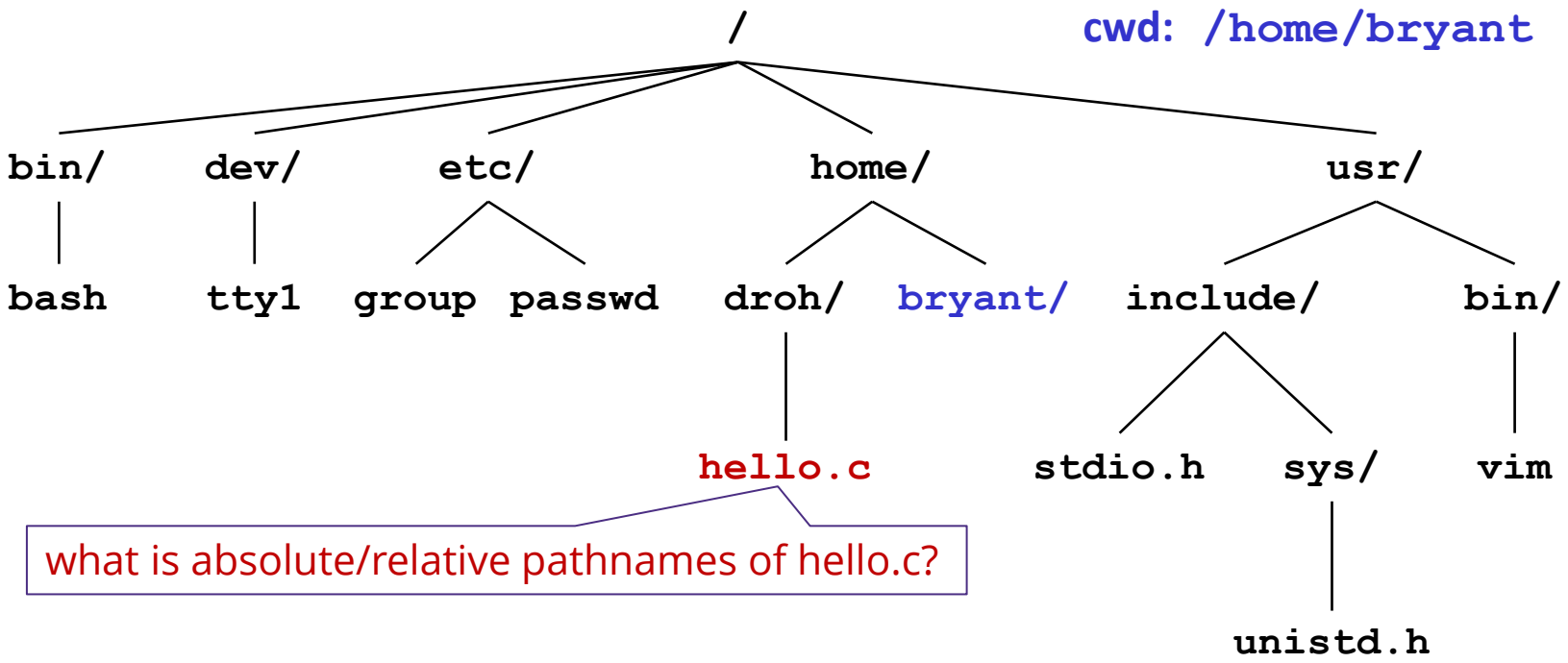
# Linux File System

- Files are organized as a hierarchy anchored by root directory (/)
  - A ***directory*** consists of an array of links, each link maps a ***filename*** to a file
  - Each directory has at least 2 links: to itself (.) & to the parent directory (..)



# Pathnames

- Denote the locations of files in the namespace hierarchy
  - ***Absolute pathname*** denotes path from root
  - ***Relative pathname*** denotes path from current working directory (`cwd`)
  - Kernel maintains current working directory for each process



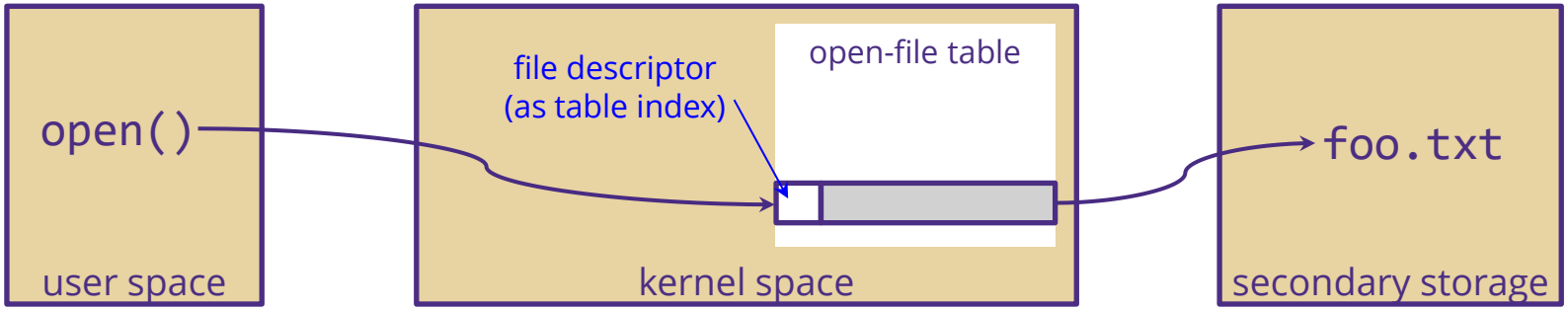
# File I/O Operations

- C programs usually use `stdio` package for file I/O.
- Library functions layered on top of I/O system calls
  - We presume understanding of `stdio` → focus on system calls.

| System calls | Library functions |
|---|---|
| `file descriptor (int)` | `file stream (FILE *)` |
| `open(), close()` | `fopen(), fclose()` |
| `lseek()` | `fseek(), ftell()` |
| `read()` | `fgets(), fscanf(), fread(), …` |
| `write()` | `fputs(), fprintf(), fwrite(), …` |

# File Descriptors & Open-file Table

- Linux requires that a file must be opened before its first use
  - Linux kernel handle opened files via their descriptors (small integers) and keeps a small table containing information about all open files.
  - Avoid constant search of the directory every time an operation is required.




- Each Linux shell process begins life with 3 open files

| FD | Purpose | POSIX name | *stdio* stream |
|---|---|---|---|
| 0 | Standard input | STDIN_FILENO | *stdin* |
| 1 | Standard output | STDOUT_FILENO | *stdout* |
| 2 | Standard error | STDERR_FILENO | *stderr* |

# Opening Files

- Informs the kernel that you are getting ready to access that file

```
#include <sys/stat.h>
#include <fcntl.h>
int fd;   /* file descriptor */

if ((fd = open("/etc/hosts", O_RDONLY)) < 0) {
   perror("open");
   exit(1);
}
```

- Returns the file descriptor, normally a small nonegative integer
  - Guaranteed to be lowest available FD
  - `fd == -1` indicates that an error occurred

# Closing Files

- Informs the kernel that you are finished accessing that file

```
#include <sys/stat.h>
#include <fcntl.h>
int fd;     /* file descriptor */
int retval; /* return value */

if ((retval = close(fd)) < 0) {
   perror("close");
   exit(1);
}
```

- Closing an already closed file is a recipe for disaster in threaded programs (more on this later)
  - **Always** releases FD, even on failure return.
- Always check return codes

# Reading Files

- Copies bytes from the current file position to memory, and then updates file position

```
#include <unistd.h>
char buf[512];
int fd;        /* file descriptor */
int nbytes;    /* number of bytes read */

/* Open file fd ...  */
/* Then read up to 512 bytes from file fd */
if ((nbytes = read(fd, buf, sizeof(buf))) < 0) {
   perror("read");
   exit(1);
}
```

- Returns number of bytes read from file `fd` into `buf`
  - `nbytes < 0` indicates that an error occurred
  - ***Short counts*** (`nbytes < sizeof(buf)`) are possible and are not errors!

# Writing Files

- Copies bytes from memory to the current file position, and then updates current file position

```
#include <unistd.h>
char buf[512];
int fd;       /* file descriptor */
int nbytes;   /* number of bytes read */

/* Open the file fd ... */
/* Then write up to 512 bytes from buf to file fd */
if ((nbytes = write(fd, buf, sizeof(buf)) < 0) {
   perror("write");
   exit(1);
}
```

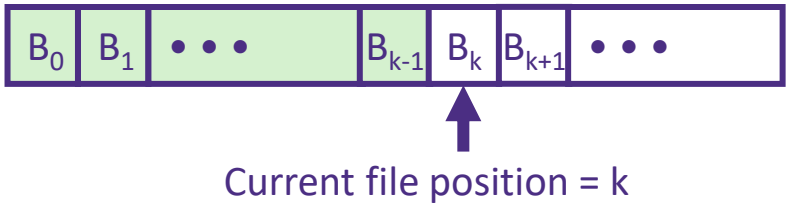
- Returns number of bytes written from `buf` to file `fd`
  - `nbytes < 0` indicates that an error occurred
  - As with reads, short counts are possible and are not errors!

# On Short Counts

- Short counts can occur in these situations:
  - Encountering end-of-file (EOF) on reads
  - Reading text lines from a terminal
  - Reading and writing network sockets
- Short counts never occur in these situations:
  - Reading from disk files (except for EOF)
  - Writing to disk files
- Best practice is to always allow for short counts.

# Seeking in a File

- Moves the current read/write position within an open file
  - E.g., media player seeks position in a video file




```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int fd;        /* file descriptor */

/* Open the file fd ... */
/* Then move to the 10th byte from the beginning */
if (offset = lseek(fd, 10, SEEK_SET)) < 0) {
   perror("seek");
   exit(1);
}
```

# File Metadata Operations

> Per-file metadata is data about the file data maintained by kernel

- accessed by users with the `stat`/`fstat` functions

```
/* Metadata returned by the stat and fstat functions */
struct stat {
    dev_t         st_dev;      /* Device */
    ino_t         st_ino;      /* inode */
    mode_t        st_mode;     /* Protection and file type */
    nlink_t       st_nlink;    /* Number of hard links */
    uid_t         st_uid;      /* User ID of owner */
    gid_t         st_gid;      /* Group ID of owner */
    dev_t         st_rdev;     /* Device type (if inode device) */
    off_t         st_size;     /* Total size, in bytes */
    unsigned long st_blksize;  /* Blocksize for filesystem I/O */
    unsigned long st_blocks;   /* Number of blocks allocated */
    time_t        st_atime;    /* Time of last access */
    time_t        st_mtime;    /* Time of last modification */
    time_t        st_ctime;    /* Time of last change */
};
```

# Accessing Directories

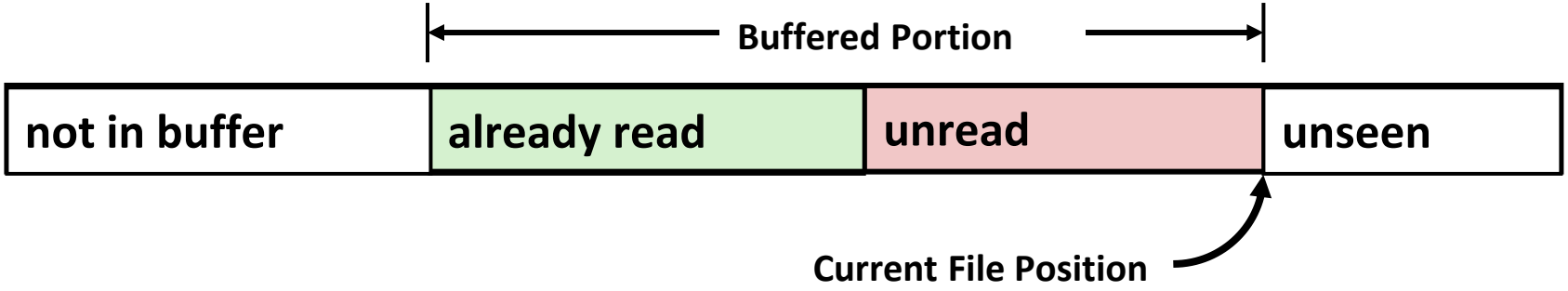
- Only recommended operation on a directory: read its entries
  - `dirent` structure contains information about a directory entry
  - DIR structure contains information about the directory while walking through it

```
#include <sys/types.h>
#include <dirent.h>

{
  DIR *directory;
  struct dirent *de;
  ...
  if (!(directory = opendir(dir_name)))
      error("Failed to open directory");
  ...
  while (0 != (de = readdir(directory))) {
      printf("Found file: %s\n", de->d_name);
  }
  ...
  closedir(directory);
}
```

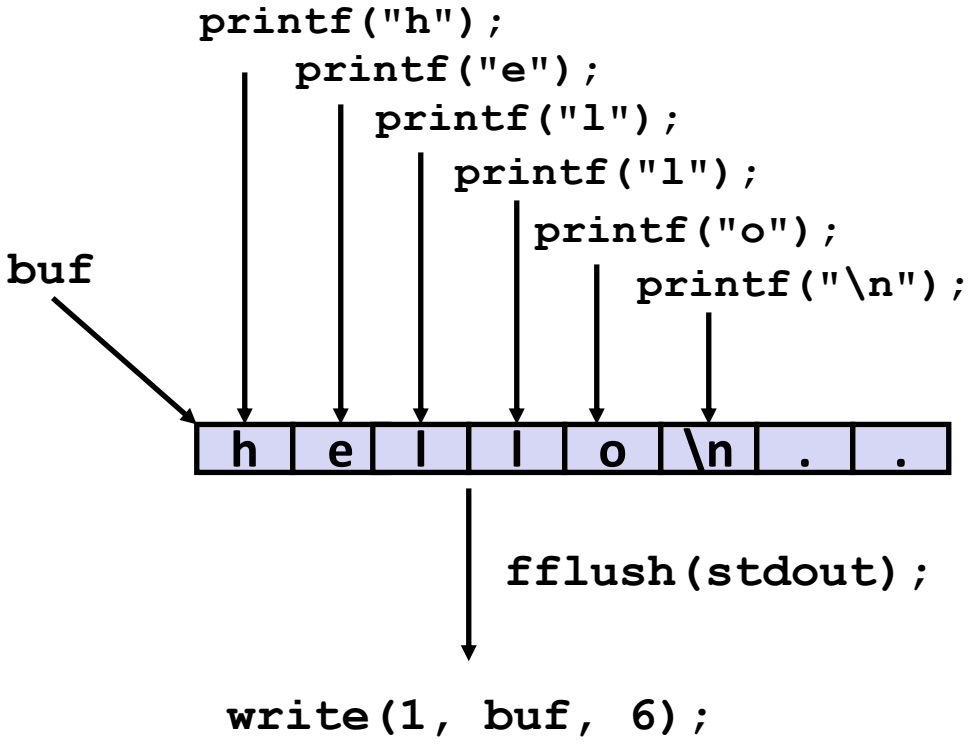
# Buffered I/O

- Applications often read/write one character at a time
  - `getc, putc, ungetc`
  - `gets, fgets`: read line of text one character at a time, stopping at newline
- Implementing via Linux system calls expensive
  - > 10,000 clock cycles
- Solution: buffered read
  - Use `read` system call to grab block of bytes
  - User input functions take one byte at a time from buffer, refill buffer when empty



# Buffering in Standard I/O

> Standard I/O functions use buffered I/O

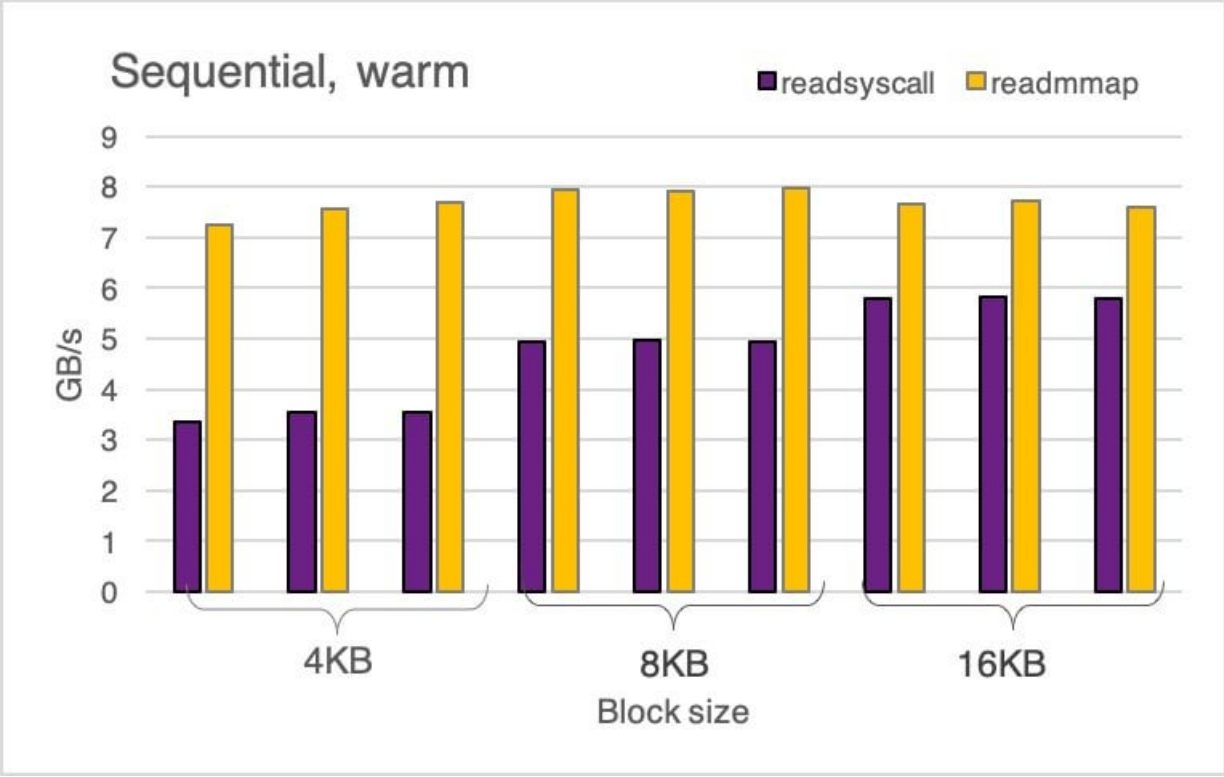



> Buffer flushed to output fd on “\n” by calling to `fflush`

# Memory-mapping I/O

> Maps a file into memory, so it can be accessed as a memory array.

- Implemented in the `mmap` function
- Only loads the required parts into RAM via on-demand loading (paging)



# Buffered vs. Memory-mapping I/O

> Comparison summary

| Feature | Buffered I/O | Memory-mapping I/O |
|---|---|---|
| ***Performance*** | Good for sequential access | Great for random access, eliminates extra copying |
| ***System Calls*** | Requires explicit `read()` and `write()` calls | Implicit—accessing memory triggers page loads |
| ***Memory Usage*** | Uses a kernel buffer first, then copy to a user-space buffer. | Directly maps file into memory |
| ***Best Use Case*** | Sequential reads/writes, small files | Large files, random access, shared memory by processes |
| ***Programming complexity*** | Simple and widely used | More complex, needs careful handling |

# NEXT LECTURE

[Flipped class] Working with processes

- Pre-class
  - Study pre-class materials on Canvas
- In class
  - Reinforcement/enrichment discussion
- Post class
  - Homework
  - Preparation for Lab 1
  - Consultation (if needed)